CSE508 Network Security

More on the Web PKI:
Certificate Transparency and ACME

Slides contributed by Ka Lok Wu

Acknowledgement

Some of the contents in this slide deck are adapted from:

- Prof. Sze Yiu Chau (Department of Information Engineering, the Chinese University of Hong Kong)
- RFC 8555: Automated Certificate Management Environment (ACME)[1]
- RFC 6962: Certificate Transparency[2]
- RFC 9162: Certificate Transparency Version 2.0[2]

[1] Proposed standard
[2] Experimental standard

Introduction

- Certificates are issued by CAs

Introduction

- Certificates are issued by CAs
- CAs can authenticate the certificate requestor by domain validation (DV), organization validation (OV) or extended validation (EV)

Introduction

- Certificates are issued by CAs
- CAs can authenticate the certificate requestor by domain validation (DV), organization validation (OV) or extended validation (EV)
  - Is there a easier way to obtain certificates?

Introduction

- Certificates are issued by CAs
- CAs can authenticate the certificate requestor by domain validation (DV), organization validation (OV) or extended validation (EV)
  - Is there a easier way to obtain certificates?
- What is stopping the CAs from issuing fraudulent certificates?

Introduction

On 15th March 2011, a RA partner of the Comodo CA suffered an internal security breach (Comodo incident report). The attacker used the RA's account with Comodo to cause 9 fraudulent certificates to be issued.

The domain names of the certificates were as follows:

- addons.mozilla.org
- login.live.com
- mail.google.com
- www.google.com
- login.yahoo.com (x3)
- login.skype.com
- global trustee

These certificates were issued between 6 and 8pm on 15th March from the UTN-UserFirst-Hardware root, without the use of an intermediate certificate. The attack was discovered almost immediately by an internal Comodo check, and the certificates were revoked (via both CRL and OCSP). The compromised RA account has been suspended.

Introduction

On 15th March 2011
report). The attacke

The domain names

- addons.mozilla.
- login.live.com
- mail.google.com
- www.google.co
- login.yahoo.com
- login.skype.com
- global trustee

These certificates w
without the use of a
Comodo check, and
has been suspende

do incident
issued.

ware root,
an internal
A account

Maintaining digital certificate security

July 8, 2014

Posted by Adam Langley, Security Engineer

On Wednesday, July 2, we became aware of unauthorized digital certificates for several Google domains. The certificates were issued by the National Informatics Centre (NIC) of India, which holds several intermediate CA certificates trusted by the Indian Controller of Certifying Authorities (India CCA).

The India CCA certificates are included in the Microsoft Root Store and thus are trusted by the vast majority of programs running on Windows, including Internet Explorer and Chrome. Firefox is not affected because it uses its own root store that doesn't include these certificates.

We are not aware of any other root stores that include the India CCA certificates, thus Chrome on other operating systems, Chrome OS, Android, iOS and OS X are not affected. Additionally, Chrome on Windows would not have accepted the certificates for Google sites because of public-key pinning, although misissued certificates for other sites may exist.

Introduction

- Certificates are issued by CAs
- CAs can authenticate the certificate requestor by domain validation (DV), organization validation (OV) or extended validation (EV)
  - Is there a easier way to obtain certificates?
- What is stopping the CAs from issuing fraudulent certificates?
- CRL (and OCSP) is not the solution: the certificates are revoked by the very CA that issues it!

Introduction

- Certificates are issued by CAs
- CAs can authenticate the certificate requestor by domain validation (DV), organization validation (OV) or extended validation (EV)
  - Is there a easier way to obtain certificates?
- What is stopping the CAs from issuing fraudulent certificates?
- CRL (and OCSP) is not the solution: the certificates are revoked by the very CA that issues it!
- What if an attacker is able to compromise a (benign) CA and issue fraudulent certificates?
- How do we detect them?

Introduction

- Certificates are issued by CAs
- What if we force all CAs to reveal all the certificates they issue into a list, and reject any certificate that is not in the list?
- What is stopping the CAs from issuing fraudulent certificates?
- CRL (and OCSP) is not the solution: the certificates are revoked by the very CA that issues it!
- What if an attacker is able to compromise a (benign) CA and issue fraudulent certificates?
- How do we detect them?

Introduction

- Certificates are issued by CAs
- What if we force all CAs to reveal all the certificates they issue into a list, and reject any certificate that is not in the list?
- What is stopping the CAs from issuing fraudulent certificates?
- How do we do that?
- What if an attacker is able to compromise a (benign) CA and issue fraudulent certificates?
- How do we detect them?

1. Introduction
2. Merkle Trees
   - Inclusion Proofs
   - Append-Only Merkle Trees
3. ACME
   - ACME
   - certbot
4. Certificate Transparency
   - Introduction
   - CT Logs
   - SCTs and Inclusion Proofs
   - Parties Involved
   - Problems with CT

Merkle Trees

We begin by a brief introduction to Merkle trees, which is a data structure used in Certificate Transparency.

Problem

- Consider a prover and a verifier, where the prover has a large set of data and the verifier wants to check if an element is in the set.

Problem

- Consider a prover and a verifier, where the prover has a large set of data and the verifier wants to check if an element is in the set.
- Attempt 1: Send the entire set?

Problem

- Consider a prover and a verifier, where the prover has a large set of data and the verifier wants to check if an element is in the set.
- Attempt 1: Send the entire set? Inefficient!

Problem

- Consider a prover and a verifier, where the prover has a large set of data and the verifier wants to check if an element is in the set.
- Attempt 1: Send the entire set? Inefficient!
- Attempt 2: Send the list of hashes of the elements, and ask the verifier to look up the table?

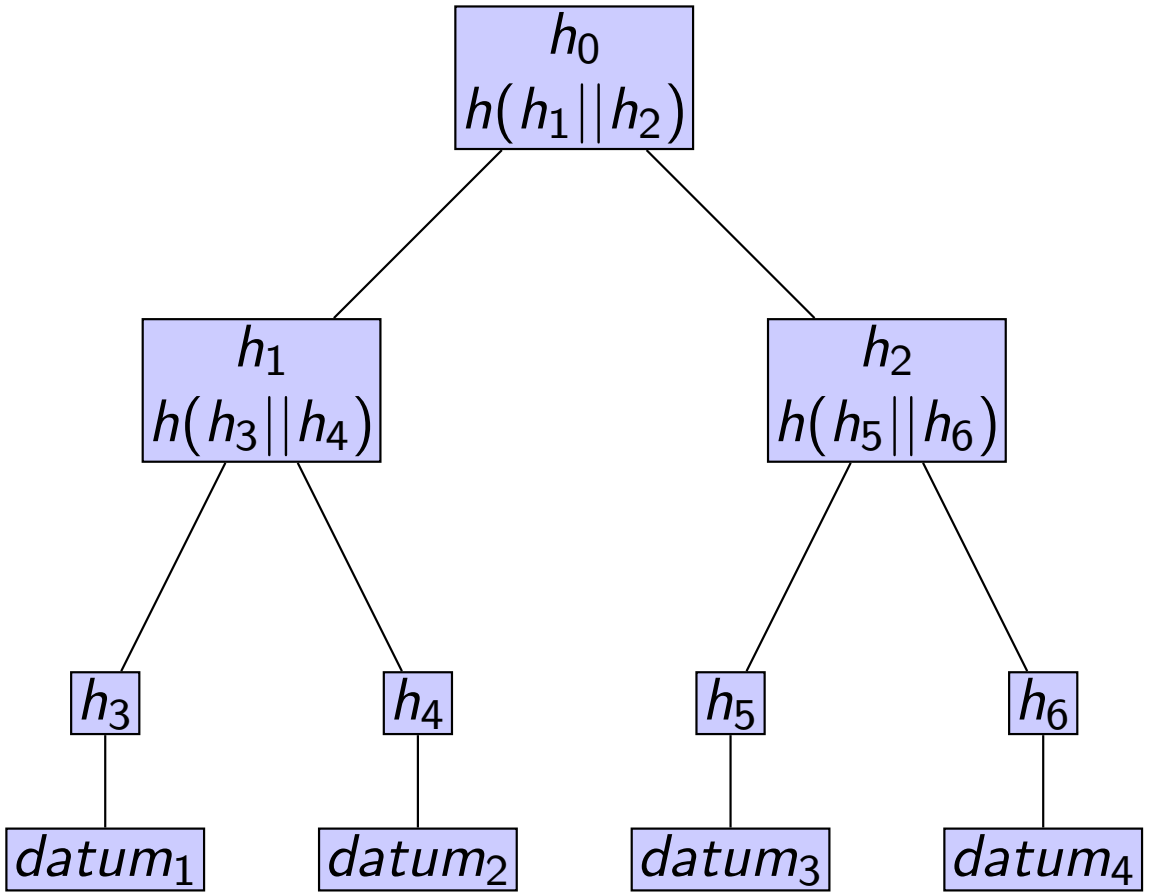
Problem

- Consider a prover and a verifier, where the prover has a large set of data and the verifier wants to check if an element is in the set.
- Attempt 1: Send the entire set? Inefficient!
- Attempt 2: Send the list of hashes of the elements, and ask the verifier to look up the table?
- Verifiers should not need to download the entire list.

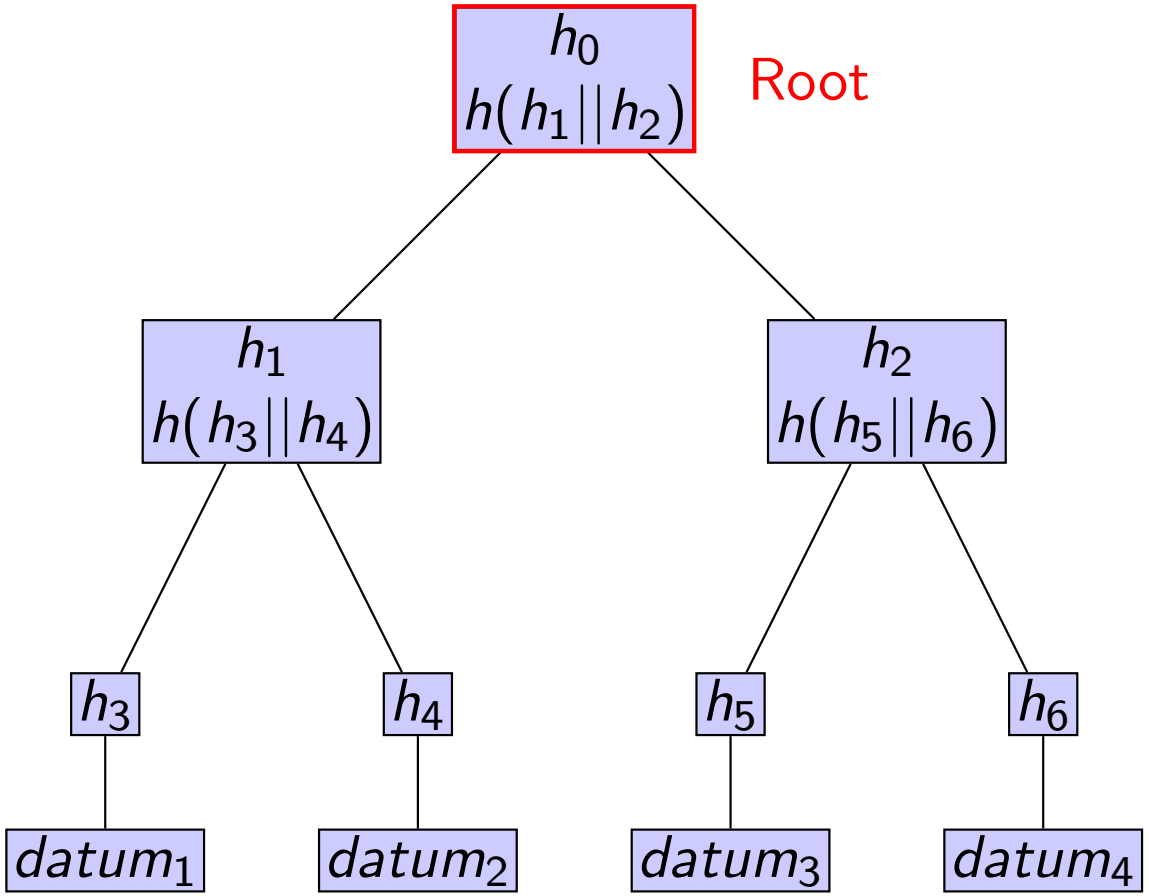
Problem

- Consider a prover and a verifier, where the prover has a large set of data and the verifier wants to check if an element is in the set.
- Attempt 1: Send the entire set? Inefficient!
- Attempt 2: Send the list of hashes of the elements, and ask the verifier to look up the table?
- Verifiers should not need to download the entire list.
- Key idea: Use a tree structure to represent the data, where the leaves are the data, every inner non-leaf stores the hash of its children, and send the *path* from the element back to the root.

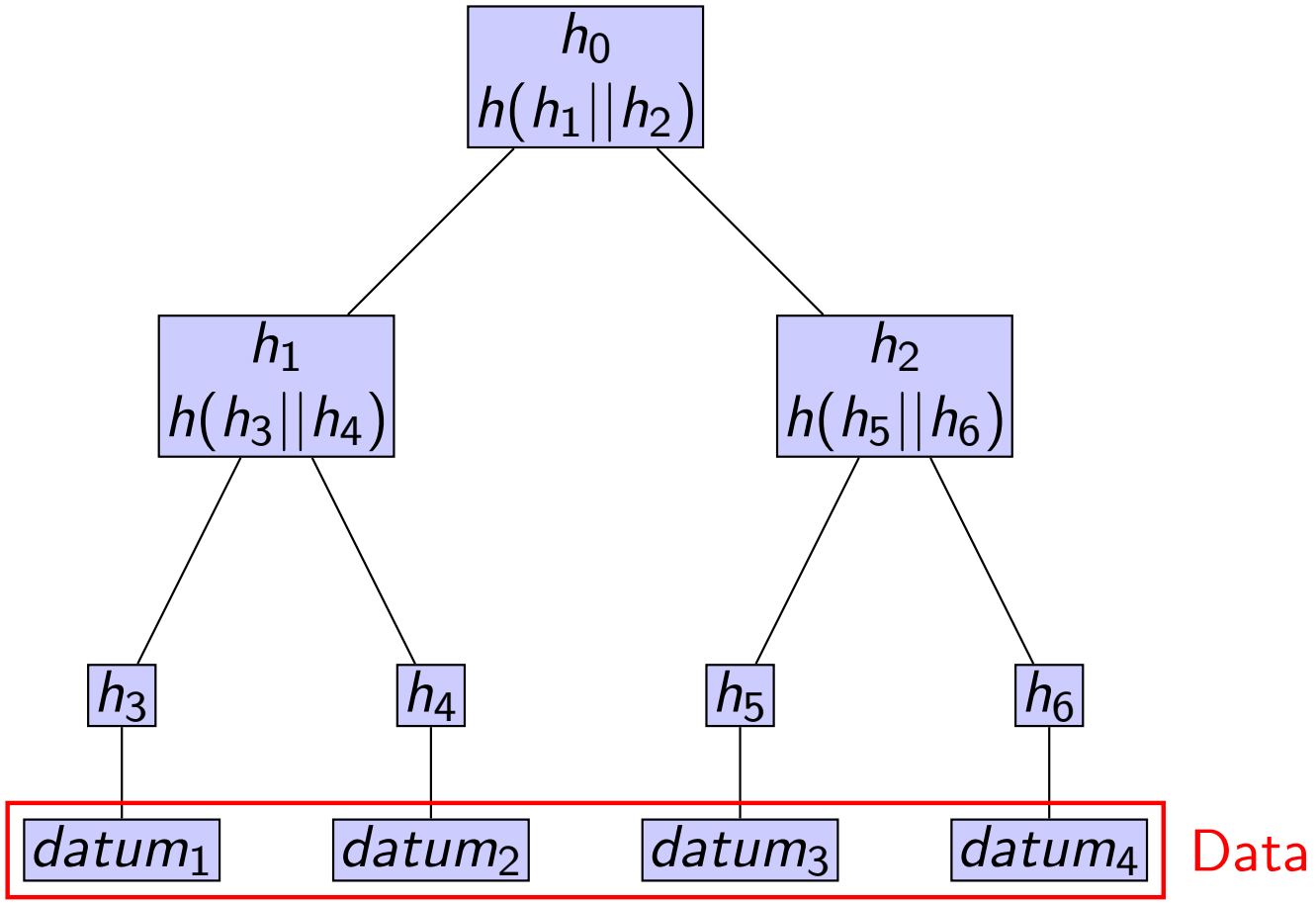
Merkle Tree


h_0
$h(h_1||h_2)$
h_1
$h(h_3||h_4)$
h_2
$h(h_5||h_6)$
h_3
h_4
h_5
h_6
$datum_1$
$datum_2$
$datum_3$
$datum_4$

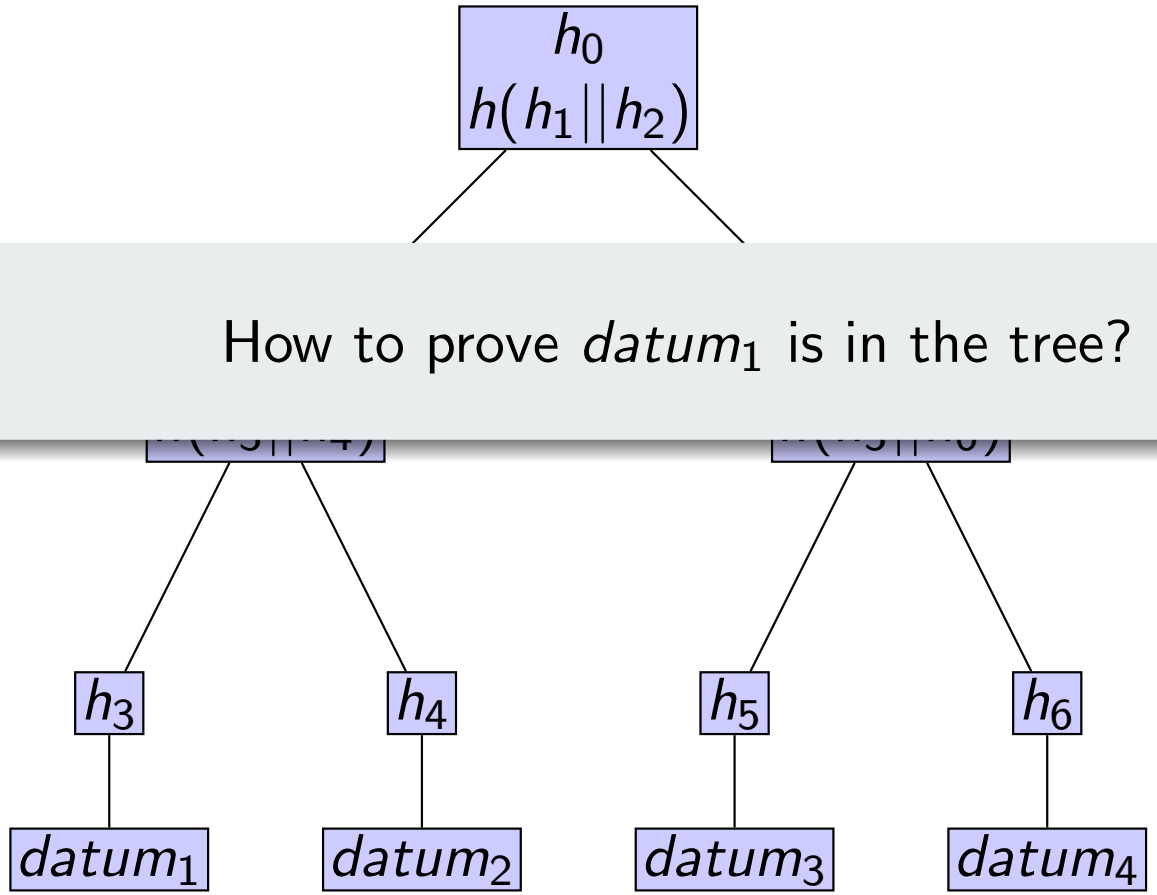
Merkle Tree


h_0
$h(h_1||h_2)$
Root
h_1
$h(h_3||h_4)$
h_2
$h(h_5||h_6)$
h_3
h_4
h_5
h_6
$datum_1$
$datum_2$
$datum_3$
$datum_4$

Merkle Tree

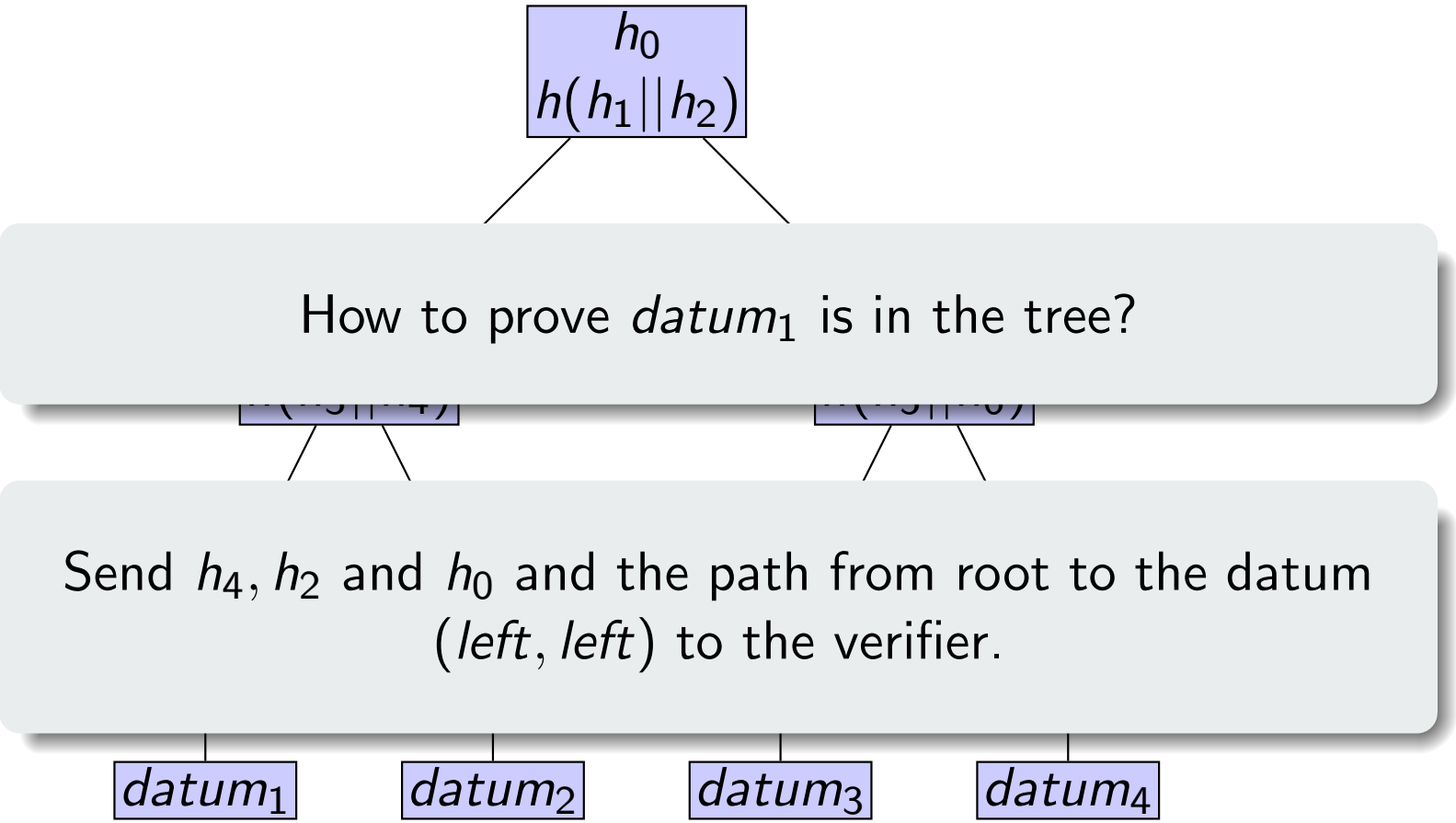

h_0
$h(h_1||h_2)$
h_1
$h(h_3||h_4)$
h_2
$h(h_5||h_6)$
h_3
h_4
h_5
h_6
$datum_1$
$datum_2$
$datum_3$
$datum_4$
Data

Merkle Tree



How to prove $datum_1$ is in the tree?

Merkle Tree


h_0
$h(h_1||h_2)$
How to prove $datum_1$ is in the tree?
Send h_4, h_2 and h_0 and the path from root to the datum $(left, left)$ to the verifier.
$datum_1$
$datum_2$
$datum_3$
$datum_4$

Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$

$datum_1$

Inclusion Proof Example

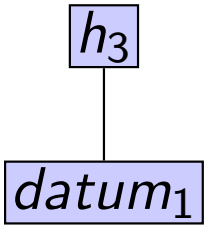
Verifier knows h_4, h_2, h_0, path is $(left, left)$

Compute $h_3 = h(datum_1)$.

$datum_1$

Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$



Inclusion Proof Example

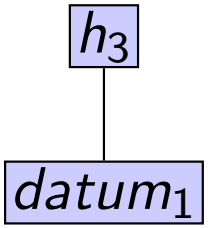
Verifier knows h_4, h_2, h_0, path is $(left, left)$

h_4 is the right sibling of h_3. Now compute
$h_1 = h(h_3||h_4)$.

$datum_1$

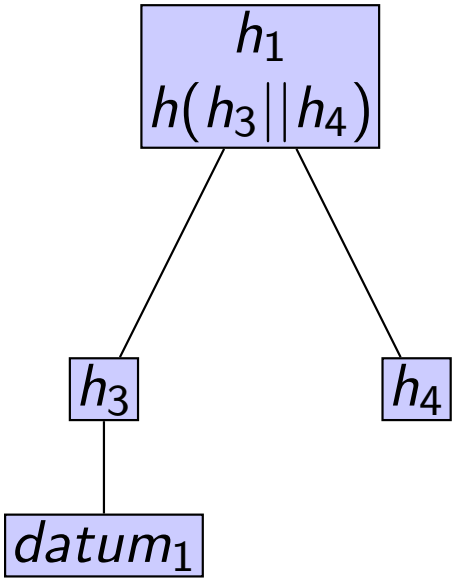
Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$



Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$



Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$

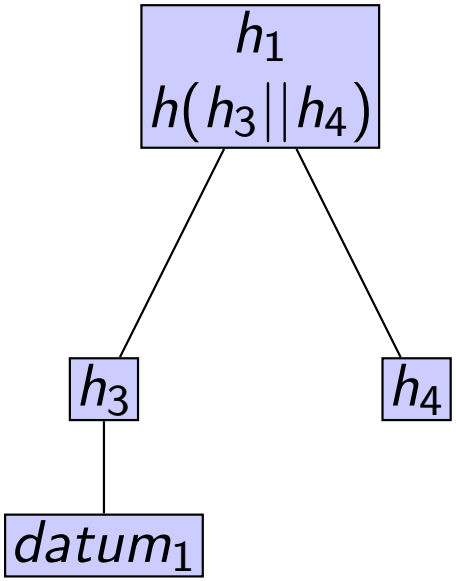
h_1
$h(h_3||h_4)$

h_2 is the right sibling of h_1. Now compute $h'_0 = h(h_1||h_2)$ and check if $h'_0 = h_0$.

$datum_1$

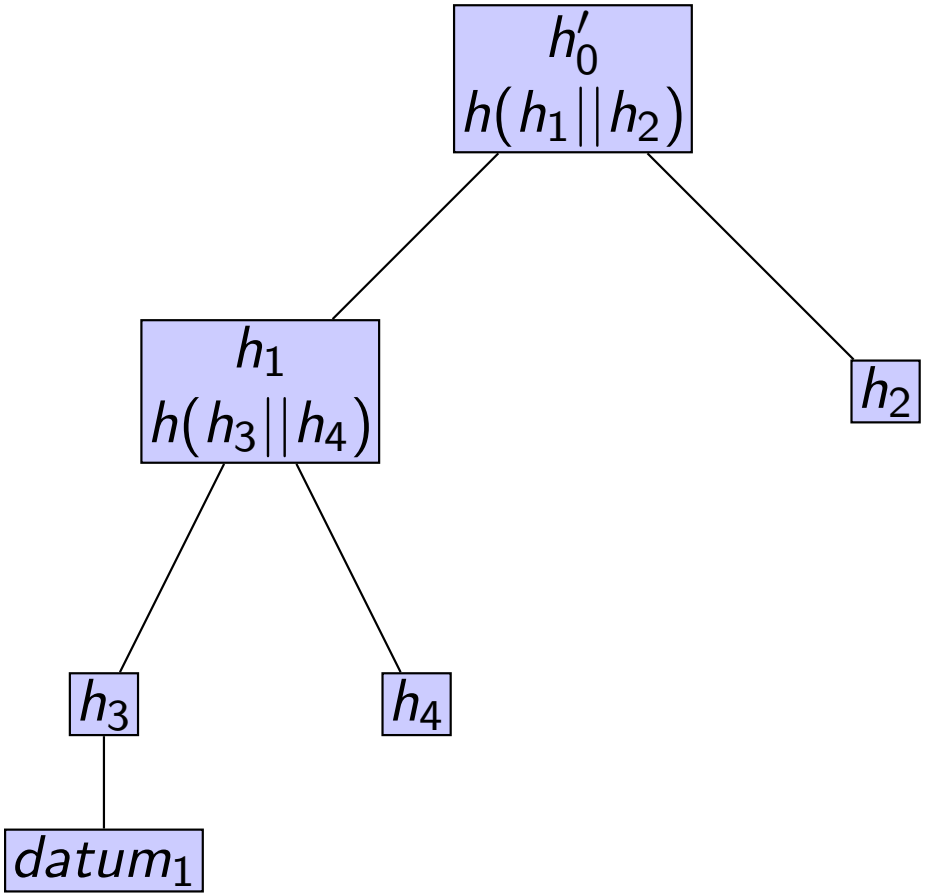
Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$



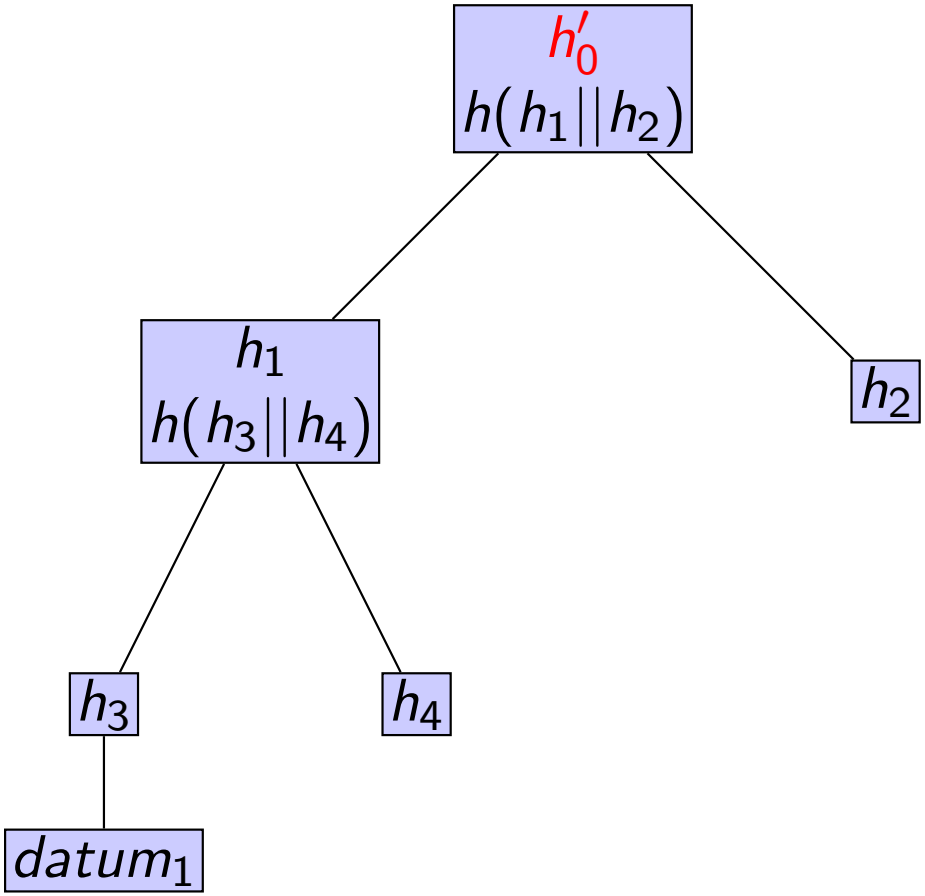
Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$



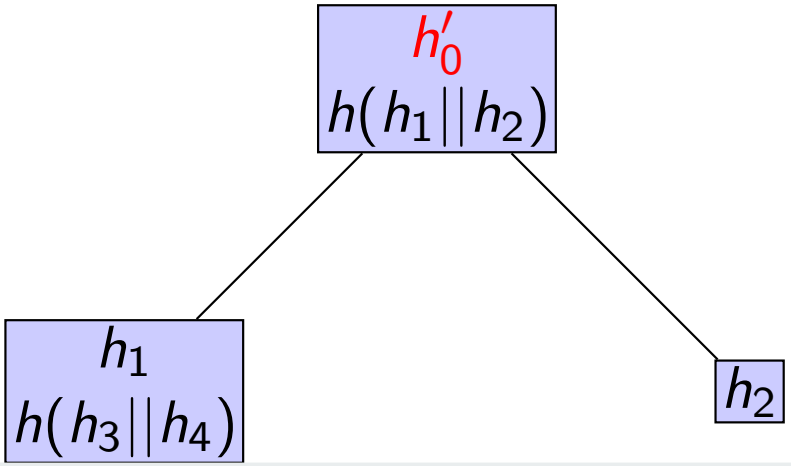
Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$

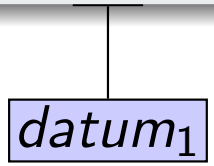


Inclusion Proof Example

Verifier knows h_4, h_2, h_0, path is $(left, left)$



We have constructed just enough of the tree to verify the membership of data!



Analysis

- How much data does the prover need to send to the verifier in an inclusion proof?
- If a collision occurs for one of the hashes (aka a second preimage collsion), what can we do?

Append-Only Merkle Trees

- We can also construct an append-only merkle tree
- In an append-only merkle tree, we can only add new leaves to the tree, but cannot remove or modify existing leaves.
- When a tree is modified, we have to recompute the hashes from the modified leaf all the way up to the root.
- How to ensure that the tree contains all the leaves that were previously added?

Append-Only Merkle Trees

- We can also construct an append-only merkle tree
- In an append-only merkle tree, we can only add new leaves to the tree, but cannot remove or modify existing leaves.
- When a tree is modified, we have to recompute the hashes from the modified leaf all the way up to the root.
- How to ensure that the tree contains all the leaves that were previously added?
- We need a *consistency proof* that given two merkle trees T_1 and T_2, where T_1 is a subtree of T_2, we can prove that T_2 contains all the leaves of T_1.

ACME

Goal

- Let's Encrypt: a CA that provides free (as in free) TLS certificates for everyone

Goal

- Let's Encrypt: a CA that provides free (as in free) TLS certificates for everyone
- You get a certificate, everyone gets a certificate!

Goal

- Let's Encrypt: a CA that provides free (as in free) TLS certificates for everyone
- You get a certificate, everyone gets a certificate!
- Getting a certificate from a CA is often manual

Goal

- Let's Encrypt: a CA that provides free (as in free) TLS certificates for everyone
- You get a certificate, everyone gets a certificate!
- Getting a certificate from a CA is often manual
- Let's Encrypt does not support revocation, and the leaf certificates they issue are only valid for 90 days

Goal

- Let's Encrypt: a CA that provides free (as in free) TLS certificates for everyone
- You get a certificate, everyone gets a certificate!
- Getting a certificate from a CA is often manual
- Let's Encrypt does not support revocation, and the leaf certificates they issue are only valid for 90 days
- Goal: automate the process of certificate issuance and renewal

ACME

- Automated Certificate Management Environment

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555
- Using JSON messages over HTTPS to communicate between the CAs and the client's servers that deploy the certs (most likely web servers)

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555
- Using JSON messages over HTTPS to communicate between the CAs and the client's servers that deploy the certs (most likely web servers)
- Accounts are bound to a public key, which is used to sign requests to the ACME server

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555
- Using JSON messages over HTTPS to communicate between the CAs and the client's servers that deploy the certs (most likely web servers)
- Accounts are bound to a public key, which is used to sign requests to the ACME server
- Challenge-Response protocol to prove control over a domain. For example:

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555
- Using JSON messages over HTTPS to communicate between the CAs and the client's servers that deploy the certs (most likely web servers)
- Accounts are bound to a public key, which is used to sign requests to the ACME server
- Challenge-Response protocol to prove control over a domain. For example:
  - HTTP: the client places a file with a specific content at a specified location

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555
- Using JSON messages over HTTPS to communicate between the CAs and the client's servers that deploy the certs (most likely web servers)
- Accounts are bound to a public key, which is used to sign requests to the ACME server
- Challenge-Response protocol to prove control over a domain. For example:
  - HTTP: the client places a file with a specific content at a specified location
  - DNS: the client creates a TXT record in the DNS zone for the domain with a specific string

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555
- Using JSON messages over HTTPS to communicate between the CAs and the client's servers that deploy the certs (most likely web servers)
- Accounts are bound to a public key, which is used to sign requests to the ACME server
- Challenge-Response protocol to prove control over a domain. For example:
  - HTTP: the client places a file with a specific content at a specified location
  - DNS: the client creates a TXT record in the DNS zone for the domain with a specific string
- Once the challenge is completed, the ACME server will issue a certificate for the domain.

ACME

- Automated Certificate Management Environment
- Proposed in RFC 8555
- Using JSON messages over HTTPS to communicate between the CAs and the client's servers that deploy the certs (most
- For example:
  - HTTP: the client places a file with a specific content at a specified location
  - DNS: the client creates a TXT record in the DNS zone for the domain with a specific string
- Once the challenge is completed, the ACME server will issue a certificate for the domain.

For HTTP challenges, if the client is already running a web server, then this can probably be done automatically!

certbot

- A python-based, free and open source client[3] for ACME to obtain certificates from Let's Encrypt
- Issuing and (in some cases) deploying certificates is fully automated
- You can even set up automatic renewal

[3] https://github.com/certbot/certbot

ACME: Domain Validation in Detail

Overall ACME Protocol Flow

1. Client generates an account key pair and registers with the ACME server
2. Client submits an order for a domain (e.g. example.com)
3. Server responds with an authorization object containing challenge options
4. Client fulfils a challenge (HTTP-01 or DNS-01) and signals readiness
5. Server verifies the challenge — authorization marked valid
6. Client submits a CSR signed with the domain key; server issues the certificate

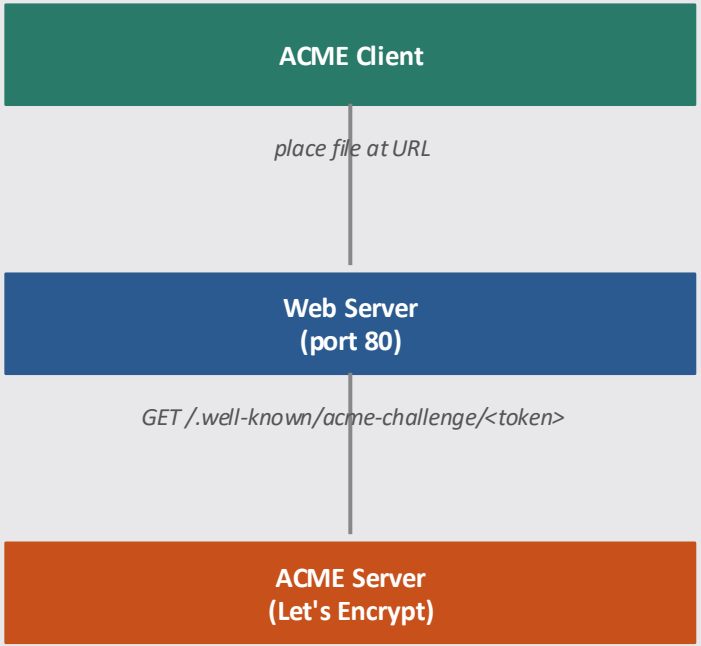
All requests are JWS-signed with the account private key; each carries a fresh server-issued nonce → replay attacks are prevented.

Key Authorization & JWK Thumbprint

- Every challenge response embeds a key authorization string
  - `key_authorization = token || "." || base64url(SHA-256(account_public_key_JWK))`
- JWK = JSON Web Key (RFC 7517) — a JSON serialization of the account public key
  - `{ "crv":"P-256", "kty":"EC", "x":"...", "y":"..." }` (keys in lexicographic order, no whitespace)
- The JWK Thumbprint (RFC 7638) is SHA-256 of that canonical JSON, base64url-encoded
  - Result is always 43 characters — compact enough for DNS TXT records
- Security role: binds the challenge to a specific ACME account
  - An attacker who intercepts a token cannot reuse it — the thumbprint of their key won't match what the server expects
- DNS-01 hashes the key authorization a second time before placing it in DNS
  - `dns_value = base64url(SHA-256(key_authorization))`

HTTP-01 Challenge — Setup

- Server issues a random token (URL-safe base64)
- Client computes the key authorization string and serves it at:
  - http://<domain>/.well-known/acme-challenge/<token>
- Must be plain HTTP (port 80), Content-Type: text/plain
- Client POSTs a signed 'ready' message to the ACME server
- Server fetches the URL and verifies:
  - ① token matches ② thumbprint matches account key



HTTP-01 Challenge — Security Properties

- Proof of domain control
  - Only the operator of the web server at that domain can place the file at the exact URL
- Replay prevention
  - Nonce in each signed ACME request; token is single-use and short-lived
- Token-forwarding attack prevented
  - Key authorization binds token to the requesting account's public key thumbprint
  - Intercepted token is useless without the matching private key
- BGP hijacking: partially mitigated by multi-perspective validation
  - Let's Encrypt queries from multiple geographically diverse vantage points
  - Attacker must hijack the victim's IP prefix simultaneously on all paths — much harder
- Limitations
  - Requires port 80 open and a running web server • Cannot issue wildcard certificates

DNS-01 Challenge — Setup

- Server again issues a random token
- Client derives the DNS value (double-hashed):
  - key_auth = token + "." + base64url(SHA-256(JWK))
  - dns_value = base64url(SHA-256(key_auth))
- Client creates a DNS TXT record:
  - _acme-challenge.<domain> IN TXT "<dns_value>"
- Server queries multiple DNS resolvers and verifies the TXT record
- Double-hashing rationale: DNS TXT length limits; hides the account key thumbprint from public DNS

Delegation Trick

CNAME _acme-challenge to a separate zone managed by a cert-management service. Primary DNS API credentials stay isolated — only the delegated zone needs write access.

Wildcard Support

DNS-01 is the only challenge type that supports wildcard certificates (*.example.com).

DNS-01 Challenge — Security Properties

- Stronger proof of domain control than HTTP-01
  - Requires write access to the DNS zone — not just a running web server
- Works behind NAT or firewalls — no inbound HTTP port required
- BGP hijacking is harder to exploit
  - DNS queries fan out to many resolvers globally; no single HTTP endpoint to intercept
  - Attacker would need to poison resolvers across many independent networks simultaneously
- New risk: DNS provider API key exposure
  - Automation requires credentials to the DNS provider API
  - Mitigation: scope API key to _acme-challenge subdomain only (if provider supports it)
  - Or delegate _acme-challenge via CNAME to an isolated zone (see previous slide)
- Account-binding: same key authorization mechanism prevents token forwarding

HTTP-01 vs DNS-01 — Comparison

Property	HTTP-01	DNS-01
Web server on port 80 needed	Yes	No
Wildcard certificate support	No	Yes
Works behind NAT / firewall	No	Yes
Automation complexity	Low	Medium
DNS API credential risk	None	Moderate
BGP hijacking resistance	Weaker (single endpoint)	Stronger (global DNS)
Token-forwarding attack prevented	Yes — JWK thumbprint	Yes — JWK thumbprint
Replay attack prevented	Yes — nonce per request	Yes — nonce per request

ACME Domain Validation — Summary

- ACME uses a challenge-response protocol to prove the requester controls the domain
- All ACME messages are JWS-signed with the account private key + fresh nonce → replay-safe
- Key authorization string = token + JWK Thumbprint of account public key
  - Binds challenge to a specific account; prevents token-forwarding attacks
- HTTP-01: client serves the key authorization at a well-known HTTP URL
  - Simple to automate; cannot issue wildcards; vulnerable to single-path BGP hijacks
- DNS-01: client provisions a TXT record at _acme-challenge.<domain>
  - Supports wildcards; stronger BGP resistance; requires DNS API access
- Multi-perspective validation (Let's Encrypt) further hardens HTTP-01 against BGP hijacks
- Next: Certificate Transparency — ensuring all issued certificates are publicly logged

Certificate Transparency

Certificate Transparency

- Certificate Transparency (CT) is a system that provides a **public** log of **all** certificates issued by public CAs
- Proposed by Google
- The goal of CT is to detect and prevent the issuance of fraudulent certificates by CAs
- When a CA issues a certificate or when a party obtains a certificate, they submit the certificate to one or more CT logs
- The log operators may also misbehave, so we need watchers that watches the watchers
- The log is an append-only Merkle tree
- The head is signed, and is called **Signed Tree Head** (STH).

Incentives to Implement CT

- Why would CAs want to submit certificates to CT logs?

Incentives to Implement CT

- Why would CAs want to submit certificates to CT logs?
- Google Chrome would reject your certificates if it is not logged!

Incentives to Implement CT

- Why would CAs want to submit certificates to CT logs?
- Google Chrome would reject your certificates if it is not logged!
- Firefox (desktop) and Safari also do the same.

Incentives to Implement CT

- Why would CAs want to submit certificates to CT logs?
- Google Chrome would reject your certificates if it is not logged!
- Firefox (desktop) and Safari also do the same.
- Essentially, by using market shares to bully CAs.

Flow of CT

- Anyone can submit a certificate to a CT log
- As long as the certificate is
  1. properly signed
  2. contains the chain of trust, and
  3. the root CA is acceptable to the log
- CAs can also prepare specicial precertificates to submit to CT logs prior to issuance
- After the certificate is included in a CT log, everyone can ask to see the inclusion proof of the certificate in the log

Signed Certificate Timestamps (SCTs)

- When a party submits a certificate to a CT log, the log will not immediately append the certificate to the log
- Instead, the log merely promises to include the certificate in the log within the maximum merge delay(MMD), usually 24 hours
- The log will generate a Signed Certificate Timestamp (SCT) for the certificate as a promise
- the SCT includes, among other things,
  1. the entry id
  2. a timestamp at which the certificate is accepted, and
  3. a signature of the submitted certificate signed by the public key of the log server.

Checking Inclusion Proofs?

- When a browser receives a server certificate, how does it check whether the certificate is included in a CT log?

Checking Inclusion Proofs?

- When a browser receives a server certificate, how does it check whether the certificate is included in a CT log?
- Browsers can request an inclusion proof from the CT log server.

Checking Inclusion Proofs?

- When a browser receives a server certificate, how does it check whether the certificate is included in a CT log?
- Browsers can request an inclusion proof from the CT log server.
- The browser can verify the inclusion proof, but this is usually not done
  -

Checking Inclusion Proofs?

- When a browser receives a server certificate, how does it check whether the certificate is included in a CT log?
- Browsers can request an inclusion proof from the CT log
- y
  not done
  -

Is there are problem with this approach?

Checking Inclusion Proofs?

- When a browser receives a server certificate, how does it check whether the certificate is included in a CT log?
- Browsers can request an inclusion proof from the CT log server.
- The browser can verify the inclusion proof, but this is usually not done
  - Speed.
    - The server needs to send the inclusion proof to the browser
    - While the inclusion proof is already made small, it still adds latency to the TLS handshake.

Checking Inclusion Proofs?

- When a browser receives a server certificate, how does it check whether the certificate is included in a CT log?
- Browsers can request an inclusion proof from the CT log server.
- The browser can verify the inclusion proof, but this is usually not done
  - Speed.
    - The server needs to send the inclusion proof to the browser
    - While the inclusion proof is already made small, it still adds latency to the TLS handshake.
  - Privacy! The browser essentially reveals what domain it is visiting to the CT log

Checking Inclusion Proofs?

- When a browser receives a server certificate, how does it check whether the certificate is included in a CT log?
- Browsers can request an inclusion proof from the CT log server.
- The browser can verify the inclusion proof, but this is usually not done
  - Speed.
    - The server needs to send the inclusion proof to the browser
    - While the inclusion proof is already made small, it still adds latency to the TLS handshake.
  - Privacy! The browser essentially reveals what domain it is visiting to the CT log
- Instead, the server include the SCT inside the certificate[4], and the client just check the signature of the SCT.

[4] They can also include it in the TLS extension or inside OCSP stapling in the certificate.

Meaning of SCTs

- What does it mean when a certificate has SCT and is checked to be valid?

Meaning of SCTs

- What does it mean when a certificate has SCT and is checked to be valid?
- It only means that the certificate was submitted to a CT log

Meaning of SCTs

- What does it mean when a certificate has SCT and is checked to be valid?
- It only means that the certificate was submitted to a CT log
- It also does not mean that the certificate is valid or trustworthy!

Meaning of SCTs

- What does it mean when a certificate has SCT and is checked to be valid?
- It only means that the certificate was submitted to a CT log
- It also does not mean that the certificate is valid or trustworthy!
  - Notice that a malicious CA can still issue a fraudulent certificate and submit it to a CT log, and obtain an SCT for it.

Meaning of SCTs

- What does it mean when a certificate has SCT and is checked to be valid?
- It only means that the certificate was submitted to a CT log
- It also does not mean that the certificate is valid or trustworthy!
  - Notice that a malicious CA can still issue a fraudulent certificate and submit it to a CT log, and obtain an SCT for it.
  - It just means that the owner of the certificate would be able to detect the fraudulent certificate, if they were indeed monitoring the logs for their domain. If they do detect it, they can try to take action like revocation.

Meaning of SCTs

- ed
- Who watches the CT logs to make sure that the logs behave correctly?
- It also does not mean that the certificate is valid or trustworthy!
  - Notice that a malicious CA can still issue a fraudulent certificate and submit it to a CT log, and obtain an SCT for it.
  - It just means that the owner of the certificate would be able to detect the fraudulent certificate, if they were indeed monitoring the logs for their domain. If they do detect it, they can try to take action like revocation.
- It does not mean that the certificate was actually included in the log yet!

Meaning of SCTs

- Who watches the CT logs to make sure that the logs behave correctly?
- It also does not mean that the certificate is valid or trustworthy!
- Are website owners supposed to monitor the CT logs constantly for their domains?

  monitoring the logs for their domain. If they do detect it, they can try to take action like revocation.
- It does not mean that the certificate was actually included in the log yet!

Monitors

- Monitors are third-party services that monitor CT logs
- They have a list of domains to monitor, and will periodically check the CT logs for new entries
- When a new certificate is found, the monitor will notify the domain owner about the new certificate.
- They should also fetch all the entries in the log to make sure that the root hash is consistent and the signature is valid in the STHs
- They would either keep a copy of the log locally and compute the new root hash, or rely on the consistency proofs provided by the log server to track the incremental changes in the log.

CLOUDFLARE

Hello,

Cloudflare has observed issuance of the following certificate for klwu.co or one of its subdomains:

Log date: [redacted]
Issuer: CN=R10,O=Let's Encrypt,C=US
Validity: [redacted]
DNS Names: www.klwu.co

Most certificates are trustworthy. However, if the data above is surprising to you or incorrect, visit https://developers.cloudflare.com/ssl/edge-certificates/additional-options/certificate-transparency-monitoring/#emails-to-be-concerned-about.

Note that Cloudflare issues backup certificates from a **different Certificate Authority** than your primary Universal SSL certificate. Read more at https://developers.cloudflare.com/ssl/edge-certificates/backup-certificates/.

This email was requested by one of your Cloudflare account administrators. If you would no longer like to receive it, please disable it under "Certificate Transparency Monitoring" at https://dash.cloudflare.com/?to=/:account/:zone/ssl-tls/edge-certificates#ct-alerting-card.

Thanks,
The Cloudflare Team

Auditors

- Auditors only take partial information and verify with other partial information that they have
- They can check for consistency proofs with pairs of STHs
- Given a certificate with an SCT, after the SCT timestamp + MMD (usually 24 hours), auditors can request an inclusion proof from the CT log to verify that the certificate was indeed included in the log.
- Monitors can also act as auditors

Domain Enumeration

- CT logs are public
- If someone issues a certificate for a domain, it will be logged in CT logs
- Even ones that you may not want to be known.
- How to prevent this?

[5] https://datatracker.ietf.org/doc/html/draft-strad-trans-redaction-01

Domain Enumeration

- CT logs are public
- If someone issues a certificate for a domain, it will be logged in CT logs
- Even ones that you may not want to be known.
- How to prevent this?
  - Not using TLS at all?

[5] https://datatracker.ietf.org/doc/html/draft-strad-trans-redaction-01

Domain Enumeration

- CT logs are public
- If someone issues a certificate for a domain, it will be logged in CT logs
- Even ones that you may not want to be known.
- How to prevent this?
  - Not using TLS at all?
  - Wildcard certificates?

[5] https://datatracker.ietf.org/doc/html/draft-strad-trans-redaction-01

Domain Enumeration

- CT logs are public
- If someone issues a certificate for a domain, it will be logged in CT logs
- Even ones that you may not want to be known.
- How to prevent this?
  - Not using TLS at all?
  - Wildcard certificates?
  - Change CT to allow domain redaction[5]?

[5] https://datatracker.ietf.org/doc/html/draft-strad-trans-redaction-01

Domain Enumeration

- CT logs are public
- If someone issues a certificate for a domain, it will be logged in CT logs
- Even ones that you may not want to be known.
- How to prevent this?
  - Not using TLS at all?
  - Wildcard certificates?
  - Change CT to allow domain redaction[5]?
  - Maybe run your own PKI?

[5] https://datatracker.ietf.org/doc/html/draft-strad-trans-redaction-01

crt.sh ID	Logged At ⇧	Not Before	Not After	Common Name	Matching Identities	Issuer Name
17533947063	2025-03-31	2025-03-31	2026-03-31	rt.cs.stonybrook.edu	rt.cs.stonybrook.edu	C=US, O=Internet2, CN=InCommon RSA Server CA 2
17533947415	2025-03-31	2025-03-31	2026-03-31	rt.cs.stonybrook.edu	rt.cs.stonybrook.edu	C=US, O=Internet2, CN=InCommon RSA Server CA 2
17472468826	2025-03-28	2025-03-28	2025-06-26	simba.cs.stonybrook.edu	simba.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17473583768	2025-03-28	2025-03-28	2025-06-26	simba.cs.stonybrook.edu	simba.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17469916990	2025-03-28	2025-03-28	2025-06-26	cs.stonybrook.edu	cs.stonybrook.edu www.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17469877981	2025-03-28	2025-03-28	2025-06-26	cs.stonybrook.edu	cs.stonybrook.edu www.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17457209082	2025-03-28	2025-03-28	2025-06-26	auth01.cs.stonybrook.edu	auth01.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17457217339	2025-03-28	2025-03-28	2025-06-26	auth01.cs.stonybrook.edu	auth01.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17380109097	2025-03-25	2025-03-24	2025-06-22	cicaida.cs.stonybrook.edu	cicaida.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=E6
17380287796	2025-03-24	2025-03-24	2025-06-22	cicaida.cs.stonybrook.edu	cicaida.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=E6
17360532713	2025-03-24	2025-03-21	2025-06-20	dsl.cs.stonybrook.edu	dsl.cs.stonybrook.edu	C=US, O=Google Trust Services, CN=WR3
17327864629	2025-03-22	2025-03-22	2025-06-20	server2.iotta.snia.org	snia1.fsl.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=E5
17327869670	2025-03-22	2025-03-22	2025-06-20	server2.iotta.snia.org	snia1.fsl.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=E5
17467642803	2025-03-22	2025-03-21	2025-06-20	dsl.cs.stonybrook.edu	dsl.cs.stonybrook.edu	C=US, O=Google Trust Services, CN=WR3
17239753813	2025-03-19	2025-03-19	2026-04-19	vpn.cs.stonybrook.edu	vpn.cs.stonybrook.edu	C=US, O=Internet2, CN=InCommon RSA Server CA 2
17239753666	2025-03-19	2025-03-19	2026-04-19	vpn.cs.stonybrook.edu	vpn.cs.stonybrook.edu	C=US, O=Internet2, CN=InCommon RSA Server CA 2
17211994285	2025-03-18	2025-03-18	2025-06-16	clanode.compas.cs.stonybrook.edu	clanode.compas.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R10
17155478582	2025-03-16	2025-03-16	2025-06-14	arrive.cs.stonybrook.edu	arrive.cs.stonybrook.edu graphene.cs.stonybrook.edu isc2019.cs.stonybrook.edu oscar.cs.stonybrook.edu protego.cs.stonybrook.edu spin2013.cs.stonybrook.edu www3.cs.stonybrook.edu www.pace.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17170408910	2025-03-15	2025-03-15	2025-06-13	dpc3.compas.cs.stonybrook.edu	dpc3.compas.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17121578186	2025-03-14	2025-03-14	2025-06-12	www3.cs.stonybrook.edu	arrive.cs.stonybrook.edu graphene.cs.stonybrook.edu isc2019.cs.stonybrook.edu polyglot.cs.stonybrook.edu protego.cs.stonybrook.edu spin2013.cs.stonybrook.edu www3.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17168017401	2025-03-14	2025-03-14	2025-06-12	www3.cs.stonybrook.edu	arrive.cs.stonybrook.edu graphene.cs.stonybrook.edu isc2019.cs.stonybrook.edu polyglot.cs.stonybrook.edu protego.cs.stonybrook.edu spin2013.cs.stonybrook.edu www3.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17619196066	2025-03-14	2025-03-14	2025-06-12	www3.cs.stonybrook.edu	spin2013.cs.stonybrook.edu www3.cs.stonybrook.edu www.pace.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11
17168015643	2025-03-14	2025-03-14	2025-06-12	www3.cs.stonybrook.edu	spin2013.cs.stonybrook.edu www3.cs.stonybrook.edu	C=US, O=Let's Encrypt, CN=R11

Figure: https://crt.sh/?q=cs.stonybrook.edu